



Java Archive Tool

Tim Schmitz
16.05.2026

Inhalt

Jar-Kommando	3
Optionen	3
Manifest-Datei	3
Archiv erzeugen	4
Archiv ausführen	4

Java Archive Tool

Das Java Archive Tool (jar) bündelt mehrere Java-Dateien in eine Archivdatei oder JAR-Datei mit dem Suffix `.jar`. Das Kompressionsformat basiert auf dem Zip-Format. Ein Java-Archiv kann als externe Bibliothek in dem Klassenpfad einer Anwendung eingebunden werden. Wird mit dem Java-Archiv auch eine Hauptklasse verpackt, so kann das Archiv als eigene Anwendung gestartet werden.

Jar-Kommando

Das Jar-Kommando verpackt Klassen in ein Archiv. Der Name des Archivs muss angegeben werden. Um ein neues Archiv zu erzeugen, wird das Jar-Kommando auf der Kommandozeile der Console mit der folgenden Option aufgerufen:

```
jar [Optionen] Archivdatei Manifestdatei [-C Klassen] Dateien
```

Optionen

-c

Diese Option erzeugt ein neues Archiv (create).

-f

Diese Option benennt den Namen der Archivdatei (file). Entfällt diese Option, so wird das Archiv an die Standardausgabe gesendet. Die Option `f` und der Name der Archivdatei gehören immer zusammen.

-m

Diese Option bezieht eine mitangegebene Manifestdatei mit ein.

-t

Diese Option listet das Inhaltsverzeichnis des Archivs auf.

-x

Diese Option extrahiert alle Dateien aus dem Archiv.

-C

Diese Option wechselt zeitweise in das angegebene Verzeichnis (change directory), während das Jar-Kommando ausgeführt wird. Es fügt alle Klassen des Verzeichnisses dem Archiv hinzu. Das Verzeichnis selbst wird nicht in dem Archiv angelegt. Anschließend wechselt die Ablaufkontrolle wieder in das ursprüngliche Verzeichnis, das vor Ausführung des Kommandos gültig war.

Manifest-Datei

Die Manifestdatei heißt normalerweise `MANIFEST.MF` und besitzt einen Verweis auf die Hauptklasse. Diese Information ist im Parameter `Main-Class` der Manifestdatei enthalten:

```
Main-Class: de.timdomain.application.main.Main
```

Normalerweise befindet sich die Manifest im zentralen Verzeichnis, in dem Applikationen, Extensionen, Klassenlader und Dienste konfiguriert werden. Es trägt den Namen `META-INF` und beinhaltet auch die Manifest-Datei. Das Manifest selbst besteht aus Schlüssel-Werte-Paaren.

Wenn man Werte hat, die aus mehreren Einträgen bestehen, so ist es wichtig, diese durch Leerzeichen zu trennen. So können im Klassenpfad der Manifest-Datei mehrere Bibliotheken mitgegeben werden:

```
Class-Path: lib/fop.jar
```

```
lib/avalon-framework-4.2.0.jar
...
```

(Dieses Beispiel stammt nicht aus dem Application-Projekt, sondern aus dem xdoc-Projekt selbst)

Am Schluß der Manifest-Datei muss zwingend eine Leerzeile oder eine neue Zeile stehen, da sonst das Java Archive Tool die vorhergehenden Zeilen nicht erkennt und etwa Bibliotheken nicht einbindet.

Wenn wir einen Pfad in den Klassenpfad einbinden wollen, der nicht auf eine jar-Datei verweist, sondern (wie im Beispiel) eine Konfigurationsdatei log4j2.xml im Verzeichnis properties beinhaltet, so sollte dieser Pfad am Anfang des Klassenpfades stehen:

```
Manifest-Version: 1.0
Main-Class: de.timdomain.application.control.Main
Class-Path: file:///C:/Users/test/Daten/Java/Applications/application/properties/
file:///C:/Users/test/Daten/Java/Libraries/apache-log4j-2.11.1-bin/log4j-api-2.11.1.jar
file:///C:/Users/test/Daten/Java/Libraries/apache-log4j-2.11.1-bin/log4j-core-2.11.1.jar
```

Anschließend werden die Klassen referenziert, die im Beispiel als Logging-Bibliotheken verwendet werden. Hierbei ist auf die richtige Verzeichnisreferenz zu achten. Es ist möglich, relative Pfade anzugeben:

- './' bedeutet, dass von der aktuellen JAR-Datei ein Unterverzeichnis in den Klassenpfad eingebunden werden soll
- '../' bedeutet, dass ein Verzeichnis oberhalb (bei Wiederholung) oder neben des aktuellen Verzeichnisses in den Klassenpfad eingebunden werden soll

Hierbei kann es relativ einfach zu Fehlern kommen, so dass bestimmte Bibliotheken nicht in den Klassenpfad miteingebunden werden. Daher ist es besser wie im obigen Beispiel, absolute Pfade zu verwenden. Allerdings sollte man die korrekte URL-Kodifizierung der Dateien beachten.

Archiv erzeugen

Ein Archiv wird erzeugt, indem man das Archivtool mit den entsprechenden Optionen aufruft. In dem folgenden Beispiel wird eine gepackte Applikation erzeugt und in das bereits vorhandene Verzeichnis target abgelegt. Die verwendete Manifest-Datei stammt aus dem Verzeichnis META-INF. Das Verzeichnis bin enthält eine Paketstruktur mit allen Klassen der Applikation. Die Option "-C" stellt sicher, dass die Paketstruktur übernommen wird.

```
jar cmf META-INF\MANIFEST.MF target\application.jar -C bin .
```

Anschließend kann man sich mit der Option "x" den Inhalt des gepackten Archivs auch wieder ausgeben lassen.

Archiv ausführen

Um eine Applikation als Archiv ausführen zu können, muss diese zuvor korrekt mit den benötigten Bibliotheken und der in der Manifest-Datei referenzierten Hauptklasse verpackt worden sein. Anschließend kann mit dem Java-Interpreter die Applikation als Archiv ausgeführt werden:

```
java -jar target\application.jar [Argumente]
```